Module 08 — Professional Python Deep Dive (Optional)

> **Optional enrichment** — not on the critical path. Use after core Advanced Python, Saturday labs, or interview prep.
> No full lab. Skip if you are still finishing Exceptions → Files → Logging.

Phase: Advanced Python

Previous: Optional after OOP — not on the Logging → Git path
Next: Return to the calendar: Git and GitHub after Logging, or MySQL as scheduled

---

Module Outcome

Avoid a few design traps: shared mutable defaults, HAS-A vs IS-A, a shared `pay()` contract, `__str__` vs `__repr__`, and exceptions that carry context.

You do not need this file to start Git or MySQL. Come back when a lab or interview names these traps.

---

1. Mutable default arguments

Understand

Default values are evaluated **once**, when the function is **defined**. A default `[]` is the **same list** on every call. Later calls keep leftover items.

How It Works

```
def add_skill(skill, skills=[]):
    skills.append(skill)
    return skills

print(add_skill("Python"))
print(add_skill("SQL"))
```

Expected Output:

```
['Python']
['Python', 'SQL']
```

Fix: default to `None`, create a new list inside.

```
def add_skill(skill, skills=None):
    if skills is None:
        skills = []
    skills.append(skill)
    return skills

print(add_skill("Python"))
print(add_skill("SQL"))
```

Expected Output:

```
['Python']
['SQL']
```

⚠ WATCH OUT

The same trap applies to `dict` and `set` defaults. Immutable defaults (`None`, `0`, `""`, `()`) are safe.

Try It

Call the broken `add_skill` twice, then the `None` version twice. Print all four results.

---

2. Composition before inheritance

Understand

Use inheritance when the child genuinely **is a** specialised parent. Use composition when an object **has a** collaborator it can swap. `OrderService` is not an email sender — it **has** a notifier.

How It Works

```
class EmailNotifier:
    def send(self, message):
        print(f"EMAIL: {message}")

class OrderService:
    def __init__(self, notifier):
        self.notifier = notifier

    def place_order(self, order_id):
        self.notifier.send(f"Order {order_id} placed")

OrderService(EmailNotifier()).place_order("ORD-101")
```

Expected Output:

```
EMAIL: Order ORD-101 placed
```

⚠ WATCH OUT

Forcing inheritance for HAS-A gives a rigid tree. Pass the helper in so Email can become SMS without rewriting the service.

Try It

Write `SmsNotifier` with `send`. Construct `OrderService(SmsNotifier())` and call `place_order("ORD-202")`.

---

3. Abstract base classes (recap)

Short recap of OOP Part 4. `raise` is allowed (this file sits after Exceptions).

Understand

When several classes must offer the same behaviour (`pay`), an ABC names the method. You cannot construct the ABC itself.

How It Works

```
from abc import ABC, abstractmethod

class PaymentGateway(ABC):
    @abstractmethod
    def pay(self, amount):
        """Return a payment result for the requested amount."""

class DemoGateway(PaymentGateway):
    def pay(self, amount):
        if amount <= 0:
            raise ValueError("Amount must be positive")
        return {"status": "success", "amount": amount}

print(DemoGateway().pay(499))
```

Expected Output:

```
{'status': 'success', 'amount': 499}
```

⚠️ WATCH OUT

A subclass that does not implement `pay` cannot be constructed (`TypeError`). That is the point of the contract.

---

4. `__str__` vs `__repr__`

Understand

`__str__` is for humans (`print`). `__repr__` is for developers (debug, logs). If `__str__` is missing, Python falls back to `__repr__`.

How It Works

```
class Product:
    def __init__(self, sku, name, price):
        self.sku = sku
        self.name = name
        self.price = price

    def __str__(self):
        return f"{self.name} (₹{self.price})"

    def __repr__(self):
        return f"Product({self.sku!r}, {self.name!r}, {self.price!r})"

p = Product("P1", "Mouse", 799)
print(str(p))
print(repr(p))
```

Expected Output:

```
Mouse (₹799)
Product('P1', 'Mouse', 799)
```

⚠ WATCH OUT

`!r` in an f-string uses `repr` of the value so names show quotes in `__repr__`.

---

5. Custom exceptions with context

Understand

A domain error can store fields (`sku`, `requested`, `available`) so callers see **why**, not only `"Error"`.

How It Works

```
class InsufficientStockError(Exception):
    def __init__(self, sku, requested, available):
        self.sku = sku
        self.requested = requested
        self.available = available
        super().__init__(f"{sku}: need {requested}, only {available} left")

try:
    raise InsufficientStockError("P1", 5, 2)
except InsufficientStockError as e:
    print(e)
    print("sku:", e.sku)
```

Expected Output:

```
P1: need 5, only 2 left
sku: P1
```

⚠ WATCH OUT

Always inherit `Exception`. Put extra data on `self` **and** pass a clear string to `super().__init__`.

Mini-Project (Assignment): Stock reserve + notify

Reserve stock, reject oversell with a named exception, and notify through a collaborator.

Objective

Reserve units from a product and notify after an accepted order.

Story / Scenario

A shop must not oversell. Notification is a HAS-A helper (`ConsoleNotifier`), not a parent class.

Requirements

- `Product` with name, price, and stock

- `reserve(quantity)` rejects invalid or unavailable quantities
- `InsufficientStockError` with a clear message (and useful fields)
- `ConsoleNotifier` used through composition by `OrderService`
- `__str__` or `__repr__` useful during debugging

Expected Behavior

- Reserve two from stock five → remaining three
- Oversell → clear message; stock unchanged
- Zero or negative → rejected
- Importing the file must not auto-place an order

Bonus Challenges

- `PaymentGateway` ABC + `DemoGateway` (OOP Part 4)
- Two unit tests for `reserve` — kit 26_unittest_utils

Done when

- [ ] Reserve two from five; remaining is three
- [ ] Oversell: message; stock unchanged
- [ ] Zero or negative rejected
- [ ] Import does not place an order

Interview Q&A

Q1. Why is `def f(items=[])` a bug?
The default list is created once at definition time and reused. Use `None` and create a new list inside.

Q2. Composition vs inheritance?
IS-A → inherit. HAS-A (notifier, gateway) → pass the helper in.

Q3. `__str__` vs `__repr__`?
`str` is readable for users. `repr` is unambiguous for developers.

Q4. Why put fields on a custom exception?
Callers can log `e.sku` without parsing the message string.

Cheat Sheet

```
skills=None + if None: []     → avoid mutable defaults
service(notifier)             → composition (HAS-A)
ABC + @abstractmethod         → shared contract (recap)
__str__ / __repr__            → human / debug
class E(Exception): ...       → domain error with fields
```